

Lab 08: Object-Oriented Programming Paradigm

1. Objectives

- Design and implement classes, objects, methods, and attributes to model real-world entities.
- Apply the core OOP principles: encapsulation, inheritance, abstraction, and polymorphism effectively.
- Implement OOP concepts across Java, Python, and C++ programming environments.
- Solve complex, real-world problems by designing and managing interacting class hierarchies.

2. Object Oriented Programming (OOP) Paradigm

Object-oriented programming (OOP) is a computer programming model that organizes software design around data, or objects, rather than functions and logic. An object can be defined as a data field that has unique attributes and behavior.

OOP focuses on the objects that developers want to manipulate rather than the logic required to manipulate them. This approach to programming is well suited for software that is large, complex and actively updated or maintained. This includes programs for manufacturing and design, as well as mobile applications. For example, OOP can be used for manufacturing system simulation software.

The organization of an object-oriented program also makes the method beneficial for collaborative development, where projects are divided into groups. Additional benefits of OOP include code reusability, scalability and efficiency.

2.1. Structure of object-oriented programming

The structure, or building blocks, of object-oriented programming include the following:

2.1.1. Classes

A class is a blueprint or template for creating objects. It defines the Attributes (Properties) and Methods (behaviors) that the objects created from the class can have.

2.1.2. Objects

Objects are instances of a class created with specifically defined data. Objects can correspond to real-world objects or an abstract entity. When class is defined initially, the description is the only object that is defined.

2.1.3. Methods

Methods are functions that objects can perform. They are defined inside a class that describe the behaviours of an object. Each method contained in class definitions starts with a reference to an instance object. Additionally, the subroutines contained in an object are called instance methods. Programmers use methods for reusability or keeping functionality encapsulated inside one object at a time.

2.1.4. Attributes

Attributes represent the state of an object. In other words, they are the characteristics that distinguish classes. Objects have data stored in the attributes field. Class attributes belong to the class itself and are defined in the class template.

2.2. Classes and Objects Implementation

Java is a popular, high-level, object-oriented programming language that is widely used for building applications ranging from desktop software to web and mobile applications. One of Java's core strengths is its support for Object-Oriented Programming (OOP), which helps organize code into reusable, modular, and manageable pieces.

For this explanation, we will use Java for all examples so you can see OOP concepts in action.

2.2.1. Classes

```
public class Employee {  
    // Attributes (properties)  
    String name;  
    int id;  
    double salary;  
  
    // Methods (behaviors)  
    void displayInfo() {  
        System.out.println("Name: " + name + ", Salary: $" + salary);  
    }  
  
    void work() {  
        System.out.println(name + " is working...");  
    }  
}
```

In the above example, Employee is a class with three properties (color and model) and two methods (displayInfo and work).

2.2.2. Objects

```
public class Main {  
    public static void main(String[] args) {  
        // Creating an object of the Employee class  
        Employee emp = new Employee();  
    }  
}
```

```
emp.name = "Alice";    // Setting the name
emp.id = 101;          // Setting the ID
emp.salary = 50000.0;  // Setting the salary

// Calling methods on the object
emp.displayInfo();     // Output: Name: Alice, Salary: $50000.0
emp.work();            // Output: Alice is working...

// Accessing fields directly
System.out.println("Employee Name: " + emp.name); // Output: Employee
Name: Alice
    }
}
```

In this example, `emp` is an object of the `Employee` class with specific values for name, id and salary, and it can invoke the `displayInfo` and `work` methods.

3. Object-Oriented Programming (OOP) Concepts

Object-oriented programming is based on the following principles:

3.1. Encapsulation

Encapsulation is a core principle of Object-Oriented Programming that emphasizes bundling data and the methods that operate on that data within a class. According to this principle, all critical information related to an object is contained inside the object itself, and only selected information is exposed to the outside world. The internal state and implementation details of an object are kept private, preventing direct access or modification by external entities.

Access to an object's data is controlled through a well-defined set of public methods. Other objects do not have direct authority to change the internal state of a class; instead, they interact with it only through these exposed methods. This mechanism of data hiding improves program security, preserves data integrity, and prevents unintended or unauthorized modifications.

Encapsulation offers several key advantages. It enhances data integrity by restricting direct access to an object's state. It simplifies system usage by exposing only necessary operations. Additionally, it improves maintainability and reusability, as changes to the internal representation of a class can be made without affecting other parts of the system that rely on its public interface.

```
public class Employee {
    // Private attributes (data is hidden)
    private String name;
    private int id;
    private double salary;
```

```
// Public setter methods (to set values)
public void setName(String name) {
    this.name = name;
}

public void setId(int id) {
    this.id = id;
}

public void setSalary(double salary) {
    this.salary = salary;
}

// Public getter methods (to get values)
public String getName() {
    return name;
}

public int getId() {
    return id;
}

public double getSalary() {
    return salary;
}

// Methods (behaviors)
public void displayInfo() {
    System.out.println("Name: " + name + ", Salary: $" + salary);
}

public void work() {
    System.out.println(name + " is working...");
}
}
```

```
public class Main {
    public static void main(String[] args) {
        // Creating an object of the Employee class
        Employee emp = new Employee();

        // Setting values using setter methods
        emp.setName("Alice");
        emp.setId(101);
        emp.setSalary(50000.0);

        // Calling methods
    }
}
```

```
emp.displayInfo();    // Output: Name: Alice, Salary: $50000.0
emp.work();           // Output: Alice is working...

// Accessing values using getter methods
System.out.println("Employee Name: " + emp.getName());
System.out.println("Employee Salary: $" + emp.getSalary());
}
}
```

In the above example, the name, id and salary variables of the Employee class are private and can only be accessed through the public getter and setter methods. This ensures that the internal state of the object is protected and can only be modified through controlled access points.

3.2. Inheritance

Inheritance is the concept that enables a class to reuse the properties and behaviours of another class. Through inheritance, relationships between classes are established in the form of parent (base) classes and child (derived) classes. This hierarchical structure allows common logic to be defined once and shared across multiple subclasses, while still permitting specialization and extension.

By promoting code reuse, inheritance reduces redundancy and development time, while improving consistency and accuracy in software systems. It also encourages careful analysis of shared and specialized features during system design, resulting in more structured and maintainable code.

```
public class Manager extends Employee {

    double calculateBonus() {
        return salary * 0.20; // 20% bonus
    }
}
```

```
public class Main {
    public static void main(String[] args) {

        Manager mgr = new Manager();
        mgr.name = "Kevin";
        mgr.id = 1023;
        mgr.salary = 80000;

        mgr.displayInfo();
        mgr.work();
    }
}
```

```
        System.out.println("Manager Bonus: $" + mgr.calculateBonus());
    }
}
```

In this example, the Manager class inherited the properties and the methods directly from the Employee class without modification. Additionally, the Manager class defines its own method, calculateBonus(), which is specific to managerial responsibilities.

3.3. Abstraction

Abstraction is a fundamental principle of Object-Oriented Programming that focuses on exposing only the essential features of an object while concealing its internal implementation details. It allows objects to reveal mechanisms that are relevant for interaction with other objects, while hiding unnecessary or complex code. This separation between interface and implementation reduces system complexity and enhances clarity.

Through abstraction, classes or interfaces are designed to define the core behaviours and responsibilities of an object without specifying the exact implementation. These abstract representations function as blueprints, describing what an object is capable of doing rather than how those actions are carried out. The detailed implementation is delegated to derived or concrete classes, which extend and specialize the abstract definition.

Abstraction supports software development at a higher conceptual level by allowing interaction with objects based on their general behavior rather than their internal structure. This approach simplifies system design and improves readability, as developers can focus on functionality instead of implementation details. It also encourages modularity and reusability, since components can be developed and maintained independently as long as they adhere to the defined abstract contracts.

Furthermore, abstraction enhances flexibility and scalability in software systems. Modifications to the internal implementation of a class do not affect dependent code, provided the abstract interface remains unchanged. This property significantly improves maintainability and supports future extensions or enhancements with minimal impact on existing systems.

```
abstract class Employee {
    protected String name;    // Variable in abstract class
    protected int id;         // Variable in abstract class
    protected double salary;  // Variable in abstract class

    // Abstract method → no body, must be implemented by subclasses
    abstract void work();

    // Concrete method → shared by all
    void displayInfo() {
        System.out.println("Name: " + name + ", Salary: $" + salary);
    }
}
```

```
}  
}
```

```
class Developer extends Employee {  
  
    @Override  
    void work() {  
        System.out.println(name + " is writing code."); // override Abstract  
method  
    }  
}
```

```
public class Main {  
    public static void main(String[] args) {  
        Developer dev = new Developer();  
        dev.name = "Tom";  
        dev.id = 2157;  
        dev.salary = 60000;  
  
        dev.displayInfo(); // Concrete method  
        dev.work();        // Abstract method implemented in subclass  
    }  
}
```

The Developer class inherits the properties from the abstract Employee class and overrides the work() method to provide behavior specific to a developer. The displayInfo() method is inherited directly from the Employee class without modification. This shows how the subclass can implement the abstract behavior defined in the parent class while reusing the concrete methods already provided.

3.4. Polymorphism

The word polymorphism means having many forms. Polymorphism allows one interface to be used for a general class of actions. The specific action is determined by the exact nature of the situation. It allows objects to share common behaviours while exhibiting different forms, and enables a single interface or method definition to represent multiple implementations, with the program determining the appropriate behaviour at runtime based on the object's actual class. This mechanism reduces code duplication and enhances system flexibility.

Polymorphism can be broadly classified into two types: compile-time polymorphism (method overloading) and runtime polymorphism (method overriding).

3.4.1. compile-time polymorphism (method overloading)

Method overloading allows a class to have more than one method with the same name, provided their parameter lists are different. It is resolved during compile time.

```
public class Employee {
    String name;
    int id;
    double salary;

    void work() {
        System.out.println(name + " is working...");
    }

    void work(String project) {
        System.out.println(name + " is working on project: " + project);
    }
}
```

```
public class Main {
    public static void main(String[] args) {
        Employee emp = new Employee();
        emp.name = "Jeorge";
        emp.id = 1452;
        emp.salary = 71000;

        emp.work(); // Output: Jeorge is working.
        emp.work("Website Redesign"); // Output: Jeorge is working on project:
Website Redesign
    }
}
```

In this example, the Employee class defines two work() methods with different parameters, demonstrating method overloading. The first work() method prints a general message that the employee is working, while the second work(String project) method prints a message about a specific project. This shows how a class can have multiple methods with the same name but different parameters, allowing the programmer to call the appropriate method depending on the context.

3.4.2. Method Overriding (Runtime Polymorphism)

Method overriding allows a subclass to provide a specific implementation for a method that is already defined in its superclass. It is resolved during runtime.

```
public class Employee {  
    String name;  
    int id;  
    double salary;  
  
    void work() {  
        System.out.println(name + " is working...");  
    }  
}
```

```
public class Developer extends Employee {  
  
    @Override  
    void work() {  
        System.out.println(name + " is writing code."); // override method  
    }  
}
```

```
public class Main {  
    public static void main(String[] args) {  
        Developer dev = new Developer();  
        dev.name = "John";  
        dev.id = 3287;  
        dev.salary = 65000;  
  
        dev.work(); // Method override  
    }  
}
```

The Developer class inherits the properties from the Employee class and overrides the work() method to provide behavior specific to a developer. When the work() method is called on the Developer object, the overridden method in the subclass is executed, printing a developer-specific message. The Developer class uses the fields inherited from Employee (name, id, and salary) but provides its own implementation of work(), demonstrating runtime polymorphism in Java.

4. Lab Tasks

4.1. Understanding Java Implementation

Refer to the Java implementation of OOP concepts available in *Lab_08_OOP_Java_Implementation.zip* and study the implementation.

4.2. Hands-on Implementation

Implement the same concepts using Python and C++ programming languages.

4.3. Real-world Scenario Implementation

Note: Java is recommended for this task; however, you may implement it in any programming language you are comfortable with.

A) A secret service Agent has the following attributes:

- A string type variable code name ;
- string type variable real name ;
- integer type variable Id Number ;
- and an agent has a **Service Record** that contains details as follows.
- A string type variable Previous rank;
- A string type variable Current rank;
- An integer type variable years of service;
- An integer type variable Successful mission count.

Also the Service Record has the methods to increment the years of service and methods to increment the successful mission count . when these methods are called it should increment the years of service and successful mission count by one.

The agent has methods namely promote, mission success, and add experience. Promote method should change the current rank to a given higher rank . mission success method should increment the successful mission count in service record. And also add experience method should increment the years of service by one in service record.

B) A Group of terrorists are planning an attack by buying some weapons from a seller.

There are two types of terrorists as **Global Terrorists** and **Local Terrorists**.

Local terrorists have the following attributes:

- A string type variable real name;
- A string type variable local code;
- A string type variable city.

Global terrorists have attributes as follows:

- A string type variable code name;
- A string type variable country.

There are two types of weapons that they can buy as Missiles and Bombs.

A **Missile** has the following attributes:

- A string type code;
- A String type variable range;
- A string type variable type.

And a **Bomb** has attributes as:

- A String type variable category;
- A float type variable duration which represents the time to activate the bomb.

Both bomb and missile have a common deploy method but the procedure for the deployment is different. Deploy method in the bomb should print the duration which it takes to activate and the deploy method in the missile should print "**Missile ready for launch**".

Both local and global terrorists have 3 common methods: attack, buy weapon, and compromise.

The attack method of Local terrorists should print the city which he is going to attack and the attack method of a global terrorist should print the country which he is going to attack. (City and country variables represent the locations where they are going to attack).

Buy weapon method should accept a Weapon type variable as the parameter and it should automatically call the deploy method of the weapon.

The compromise method should return the real name and code name of the local and global terrorists, respectively.

The secret service agent has a method that accepts terrorist type variables as the parameter. The secret service agent can catch terrorists using his catch Terrorist method. The method should print

["Terrorist" + (the real name of local terrorist or code name of global terrorist) + "caught by " + (the code name of the agent)]

C) There is only one Seller who has one missile and one bomb to sell. The secret service has deployed one of its best agents to catch the terrorists and the agent is currently working with the seller.

The seller has a sell weapon method that accepts one terrorist type variable and one Agent type variable.

When a terrorist is going to buy a weapon first, he should give a password to the seller. The current password is "RAPTOR". If the password is incorrect, he will not be able to buy a weapon.

After entering the correct password, he can select between the bomb and missile. The terrorist can enter number 1 to buy the bomb and 2 to buy the Missile. The terrorist can buy the bomb, attack the location, and get away successfully. But when he buys the missile, he will be caught by the agent, and the agent will be promoted to 'Major' rank, his successful mission count will be increased and his experience will also be increased.

1. Implement all attributes and methods of Agent, Terrorists, and weapons by correctly following OOP concepts.
2. Create a seller class and create a bomb and a missile inside the seller class with the following attributes:

Bomb b1

category='Clemo',
duration=12.00,
type=nuclear

Missile m1

code='0001'
range='Intercontinental'

3. Implement sell weapon method which accepts 2 parameters Terrorist type variable and an Agent type variable and implement complete procedure mentioned in Part C.
4. The details of a local terrorist who is going to buy some weapons and an agent who is working with the seller are as follows.

A local terrorist L1

Real name="John Cobra",
Local code="hunter",
City=Berlin.

An Agent A1

name="James bond"
code="007"
Id Number="99"

The service records of A1 are as follows.

Previous rank=Lieutenant,
Current rank=Captain,
Years of service=10,
Successful mission count=23.

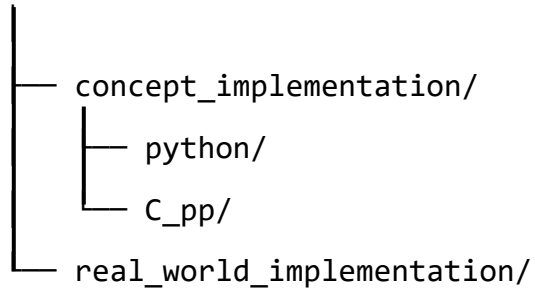
Create the agent and local terrorist objects inside your Main class and show the results of buying weapons by the terrorist

5. Submission

Create a ZIP file named **CO523_Lab08_EXXYYY.zip**, where EXXYYY represents your E-number.

The ZIP file must contain your answer scripts organized using the following folder structure:

CO523_Lab08_EXXYYY/



Upload the Zip file to the FEeLS by the given deadline.